AVALIAÇÃO DO MÓDULO 3 - BANCO DE DADOS E CONTROLE DE VERSÃO

Projeto de Extensão: Programa de Extensão UFMS Digital (95DX7.200525)

Nome completo: Matheus Leal da Gama

Disciplina: Projeto Integrador de Tecnologia da Informação II

Semestre letivo: 2026/2

Curso: Tecnologia da Informação

Público-alvo: Protetores independentes, projetos voluntários de proteção animal e pessoas interessadas em adoção responsável

Local de realização: Brasília - DF

Título da ação: PLATAFORMA WEB PARA APOIO À ADOÇÃO RESPONSÁVEL DE CÃES

Resumo: Este relatório apresenta a modelagem, a implementação SQL e o controle de versão do banco de dados da aplicação Patas de Brasília, em continuidade aos Módulos 1 e 2. A solução atende à necessidade de centralizar informações sobre cães resgatados, seus responsáveis, fotografias e mudanças de situação. O modelo relacional utiliza SQLite e mantém os cinco modelos de domínio implementados em Django, integrados ao cadastro de usuários do framework. Foram documentadas as entidades, cardinalidades, chaves, regras de integridade e decisões de normalização. O esquema foi exportado das migrations e executado por scripts SQL com dados fictícios, consultas relacionais, atualização de adoção com histórico e exclusão de uma fotografia duplicada. A execução isolada confirmou as restrições físicas, o rollback e a integridade referencial, enquanto os nove testes existentes da aplicação passaram. O código, os scripts e a documentação foram organizados em repositório GitHub, com commits descritivos por etapa. Os resultados são técnicos e preliminares, permanecendo prevista a validação com o público-alvo.

Palavras-chave: Adoção animal. Banco de dados relacional. SQLite. SQL. Controle de versão.

1 INTRODUÇÃO

O levantamento do Módulo 1 identificou que protetores independentes de Brasília mantêm informações sobre cães resgatados em conversas de WhatsApp, publicações no Instagram, documentos, planilhas e anotações. A fragmentação dificulta localizar o responsável por um animal, acompanhar seu tratamento e manter atualizada a divulgação para adoção. O

Módulo 2 materializou esses requisitos na aplicação web Patas de Brasília, utilizando Python, Django 5.2, HTML, CSS e SQLite.

Neste módulo, o foco recai sobre a organização e a manipulação persistente desses dados e sobre a rastreabilidade da evolução do projeto. Um modelo relacional permite relacionar cada cão ao seu responsável e reunir fotografias e eventos sem multiplicar os dados cadastrais. O Git registra as alterações do código e dos scripts; o GitHub disponibiliza o repositório para consulta e reprodução da atividade.

A escolha do SQLite preserva a tecnologia já adotada no MVP e permite demonstrar SQL sem instalar um servidor de banco de dados. O Django continua responsável pelas migrations e pela integração da persistência com a aplicação. O roteiro SQL deste módulo é executado em memória, com dados fictícios, de modo independente do banco local utilizado pelo sistema.

2 OBJETIVO GERAL

Consolidar e documentar o banco de dados relacional da aplicação Patas de Brasília, implementando e verificando operações SQL de criação, inserção, consulta, atualização e remoção, com gerenciamento e publicação dos artefatos por meio de Git e GitHub.

3 OBJETIVOS ESPECÍFICOS

a) Modelar as entidades, os atributos, as cardinalidades e as restrições vinculadas aos requisitos RF01 a RF08 definidos nos módulos anteriores.

b) Implementar um esquema SQL compatível com as migrations do Django e documentar as decisões de normalização e de integridade.

c) Executar inserções, consultas com relacionamentos e agregações, atualização de situação e remoção controlada de dados fictícios.

d) Verificar chaves primárias e estrangeiras, unicidade, nulabilidade, rollback e consistência dos resultados obtidos.

e) Organizar os scripts, o diagrama e as evidências em repositório GitHub, registrando commits descritivos e instruções de reprodução.

4 JUSTIFICATIVA E DELIMITAÇÃO DO PROBLEMA

A ausência de um cadastro centralizado favorece registros duplicados e informações contraditórias sobre a situação dos animais. Relacionar cães, protetores, fotografias e eventos por identificadores reduz a repetição de dados e facilita consultas como “quais cães estão sob responsabilidade de cada protetor?” e “quais animais devem aparecer na vitrine pública?”. A preservação do histórico permite consultar o andamento do atendimento e da adoção sem depender de mensagens dispersas.

O escopo permanece o dos módulos anteriores: autenticação, cadastro e divulgação de cães, identificação do responsável, contato por WhatsApp, seção de apoio com PIX e acompanhamento básico de situação. Adotantes, prontuários veterinários completos, recebimentos financeiros e contratos de adoção não foram acrescentados como entidades. A seção de apoio divulga uma chave PIX; ela não registra transações financeiras.

O SQLite é adequado à demonstração local e ao porte inicial do projeto. O crescimento do uso simultâneo poderá exigir avaliação de migração para PostgreSQL, índices adicionais, backup e armazenamento externo de mídia. O controle de versão complementa a organização dos dados ao permitir comparar alterações do código e do esquema; ele não substitui backup dos registros do banco.

5 FUNDAMENTAÇÃO TEÓRICA

A modelagem relacional organiza informações em tabelas com atributos e identificadores. Uma chave primária distingue cada registro, enquanto uma chave estrangeira estabelece a referência a outra tabela. Restrições como PRIMARY KEY, UNIQUE e NOT NULL expressam condições que o SGBD deve manter ao executar operações de escrita (SQLite, [s. d.]a). Essas restrições são relevantes para impedir que um cão seja associado a um responsável inexistente ou que um mesmo usuário receba dois perfis de protetor.

A normalização busca reduzir redundâncias e anomalias de inserção, alteração e exclusão. Na primeira forma normal, os atributos são atômicos e grupos repetidos são separados; na segunda, os atributos dependem da chave completa; na terceira, evitam-se dependências transitivas entre atributos não chave (Microsoft, [s. d.]). No projeto, dados do protetor não são copiados para cada cão, fotografias adicionais não ocupam colunas numeradas e o histórico possui uma linha por evento.

SQL reúne comandos de definição de dados, como CREATE TABLE, e de manipulação, como INSERT, UPDATE e DELETE; SELECT permite recuperar informações e combinar tabelas. A integridade referencial exige que referências não nulas apontem para registros existentes. No SQLite, a verificação de chaves estrangeiras deve ser habilitada na conexão, por exemplo com PRAGMA foreign_keys = ON (SQLite, [s. d.]b).

As migrations do Django descrevem alterações estruturais e suas dependências, permitindo reconstruir o banco a partir do código versionado (Django Software Foundation, [s. d.]a). É necessário distinguir regras físicas de validações da aplicação: choices define opções validadas pelo Django, blank trata a obrigatoriedade nos formulários e null determina a possibilidade de SQL NULL. Os comportamentos on_delete são executados pelo ORM, sem criar automaticamente ações SQL ON DELETE equivalentes (Django Software Foundation, [s. d.]b).

O Git registra versões dos arquivos em commits. A área de preparação permite selecionar o conteúdo de cada registro, enquanto diferenças e histórico ajudam a revisar as mudanças. Branches permitem linhas de desenvolvimento e merges integram seus históricos. Commits com escopo claro e mensagens explicativas facilitam a compreensão e a recuperação de versões (Chacon; Straub, 2014). No projeto, os scripts SQL e as migrations são versionados junto ao código que depende deles.

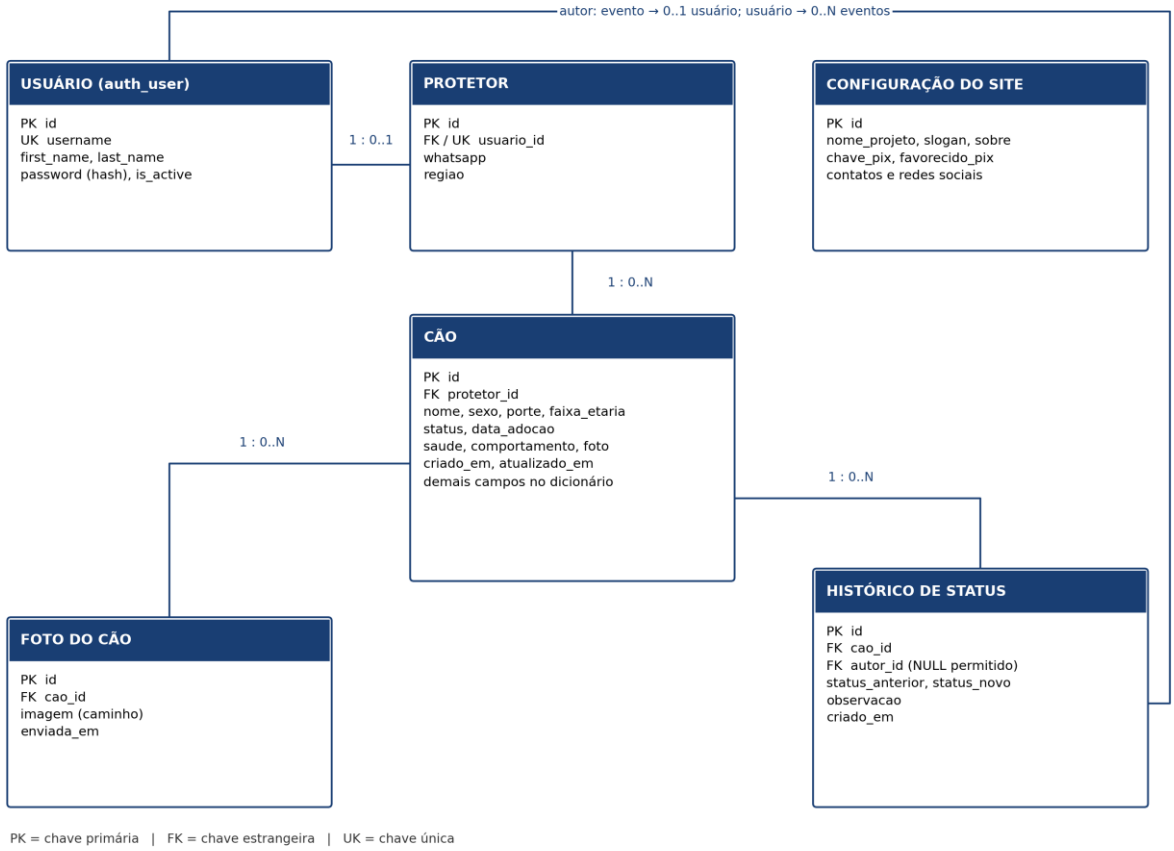
6 METODOLOGIA

O trabalho foi realizado de forma incremental: leitura dos relatórios anteriores; conferência de models.py, forms.py e migrations; elaboração do diagrama e do dicionário; exportação do DDL; escrita da carga fictícia e das operações SQL; execução das verificações; organização das evidências e elaboração deste relatório. O esquema da aplicação foi preservado para manter a continuidade do MVP.

6.1 Modelagem do Banco de Dados

O núcleo contém cinco modelos de domínio: Protetor, Cao, FotoCao, HistoricoStatus e ConfiguracaoSite. A autenticação utiliza o modelo User do Django, cuja tabela física é auth_user. A Figura 1 apresenta esse recorte com seus principais atributos; o arquivo banco_dados/dicionario.md descreve todas as colunas dos seis modelos, seus tipos físicos, nulabilidade e relacionamentos.

Figura 1 – Modelo relacional do núcleo da aplicação Patas de Brasília



Fonte: elaboração própria (2026).

Um usuário pode possuir zero ou um perfil de protetor; cada perfil pertence a exatamente um usuário. Um protetor pode ser responsável por vários cães, e cada cão possui exatamente um responsável. Um cão pode ter zero ou várias fotos adicionais e eventos de histórico. Cada evento pertence a um cão e admite zero ou um autor, pois esse vínculo pode ficar nulo. ConfiguracaoSite é independente das demais entidades.

Quadro 1 – Entidades, atributos principais e vínculo com os requisitos

Entidade / tabela	**Dados e finalidade**	**Requisitos**
User / auth_user	Identificador, usuário único, senha em hash e dados de autenticação.	RF01
Protetor / animais_protetor	Usuário único, WhatsApp e região de atuação.	RF01, RF05, RF06
Cao / animais_cao	Nome, sexo, porte, faixa etária, idade, localização, responsável, saúde, foto, situação e datas.	RF02–RF05
FotoCao / animais_fotocao	Cão, caminho da imagem adicional e data de envio.	RF02
HistoricoStatus / animais_historicostatus	Cão, situação anterior e nova, observação, autor e data.	RF03, RF08
ConfiguracaoSite / animais_configuracaosite	Nome do projeto, apresentação, chave PIX e contatos gerais.	RF06, RF07

Fonte: elaboração própria (2026).

Os atributos são atômicos no escopo do cadastro, e fotos e eventos são armazenados em tabelas próprias, atendendo à 1FN. As entidades de domínio utilizam chaves simples, sem dependências parciais, em consonância com a 2FN. A separação dos dados de usuários e protetores evita dependências transitivas no cadastro do cão, apoiando a 3FN. Idade aproximada é mantida como descrição informada pelo responsável; não equivale a uma data de nascimento.

A situação atual em Cao e as transições em HistoricoStatus atendem a finalidades distintas, mas exigem consistência: a alteração de situação deve registrar um evento. A foto principal permanece em Cao, conforme o Módulo 2; a tabela FotoCao contém a galeria adicional. O banco armazena caminhos de imagens, enquanto os arquivos ficam na mídia da aplicação.

Quadro 2 – Restrições físicas e regras da aplicação

Camada	Regra implementada
Banco SQLite	PK nas entidades; UNIQUE em Protetor.usuario_id; FKs para usuário, protetor e cão; NOT NULL nos campos não anuláveis.
Banco SQLite	data_adocao e autor_id admitem NULL. Textos opcionais são representados por string vazia. Índices das FKs são gerados pelo Django.
Django / formulários	Sexo, porte, faixa etária e situação usam choices; a data de adoção é exigida nos formulários quando o cão está adotado.
Django / ORM	PROTECT impede remover um protetor com cães; CASCADE remove fotos e eventos ao excluir o cão; SET_NULL preserva eventos quando o autor é removido.
Django / modelo	ConfiguracaoSite.save() fixa a chave em 1 para manter uma configuração pelo fluxo da aplicação.

Fonte: elaboração própria (2026).

O DDL exportado não contém CHECK de domínio para os campos de escolha nem para a regra de data de adoção. O SQLite também não impõe os limites de varchar(n) como um SGBD de tipagem rígida. As referências exportadas usam NO ACTION e verificação diferida; ao executar SQL diretamente, é preciso tratar dependentes, histórico e datas de forma explícita. Assim, a execução direta não reproduz automaticamente os comportamentos do ORM.

6.2 Implementação e Manipulação de Dados

O arquivo banco_dados/01_esquema.sql foi gerado a partir das migrations aplicadas em um banco em memória. Ele contém 14 tabelas e 19 índices explícitos, incluindo o núcleo de negócio e as estruturas de autenticação, permissões, sessões e administração do Django. O diário django_migrations e os dados de permissões foram omitidos do roteiro acadêmico. Na aplicação, a inicialização continua sendo realizada por python manage.py migrate.

O trecho a seguir mostra a tabela de protetores, preservando os nomes e as restrições físicas do esquema. O perfil utiliza uma FK única para o usuário. A cláusula DEFERRABLE INITIALLY DEFERRED permite que a checagem referencial seja concluída no COMMIT; o roteiro considera essa característica nas verificações de falha.

Listagem 1 – Trecho do esquema SQL: perfil do protetor

```
CREATE TABLE "animais_protetor" (
    "id" integer NOT NULL PRIMARY KEY AUTOINCREMENT,
    "whatsapp" varchar(20) NOT NULL,
    "regiao" varchar(80) NOT NULL,
    "usuario_id" integer NOT NULL UNIQUE
        REFERENCES "auth_user" ("id")
        DEFERRABLE INITIALLY DEFERRED
);
```

Fonte: banco_dados/01_esquema.sql, gerado das migrations do projeto (2026).

A carga 02_dados.sql insere dois usuários inativos com senha inutilizável, dois protetores, quatro cães, duas fotos adicionais, cinco eventos e uma configuração. Nomes, contatos e situações são fictícios. A configuração não contém chave PIX real, e os caminhos de imagens são ilustrativos. Não foi exportado conteúdo do banco local da aplicação.

As cinco consultas de 03_consultas.sql cobrem a vitrine pública com responsável, a contagem de cães não adotados por protetor, o histórico cronológico de um animal, a relação de adotados e o agrupamento por situação. Elas utilizam INNER JOIN, LEFT JOIN, WHERE, ORDER BY, COUNT e GROUP BY. A consulta pública seleciona campos explícitos, sem observações internas, e mantém os mesmos status públicos do Módulo 2.

Listagem 2 – Consulta dos cães da vitrine e de seus responsáveis

```
SELECT c.id, c.nome, c.status, u.first_name AS protetor,
       c.localizacao
FROM animais_cao AS c
JOIN animais_protetor AS p ON p.id = c.protetor_id
JOIN auth_user AS u ON u.id = p.usuario_id
WHERE c.status IN ('lar_temporario', 'disponivel', 'em_adocao')
ORDER BY c.nome;
```

Fonte: banco_dados/03_consultas.sql (2026).

Em 04_manipulacao.sql, uma transação insere o evento de adoção de Paçoca e atualiza sua situação, a data de adoção e a data de atualização. O INSERT do evento usa a situação anterior selecionada do próprio cão. O roteiro também remove uma fotografia adicional duplicada, preservando o cadastro do animal. A remoção atua no registro; não apaga arquivos de mídia do disco.

Listagem 3 – Atualização com histórico e exclusão de registro duplicado

```
BEGIN;
INSERT INTO animais_historicostatus
    (status_anterior, status_novo, observacao, criado_em,
     autor_id, cao_id)
SELECT status, 'adotado', 'Termo fictício assinado',
       '2026-10-15 15:00:00', 1, id
FROM animais_cao WHERE id = 1 AND status = 'disponivel';

UPDATE animais_cao
SET status = 'adotado', data_adocao = '2026-10-15',
    atualizado_em = '2026-10-15 15:00:00'
WHERE id = 1 AND status = 'disponivel';

DELETE FROM animais_fotocao
WHERE id = 2 AND cao_id = 1
  AND imagem = 'caes/extras/pacoca_duplicada.jpg';
COMMIT;
```

Fonte: banco_dados/04_manipulacao.sql (2026).

A execução completa ocorre com python banco_dados/demonstrar.py, usando somente a biblioteca padrão do Python. O programa cria um banco em memória, aplica o esquema e a carga, mostra as consultas antes e depois das mudanças e encerra com erro se alguma verificação falhar. São testados também rollback, PK duplicada, unicidade do perfil, nome nulo, referência inexistente e tentativa de remover protetor com cães. As evidências ficam registradas em banco_dados/evidencias_sql.txt.

6.3 Uso do Controle de Versão

Foi mantido o repositório já utilizado no desenvolvimento, cujo remoto origin aponta para https://github.com/Mathuesgama/projeto_rf2. O código permanece organizado no projeto config, no aplicativo animais e nas pastas templates, static e seed. A pasta banco_dados reúne os scripts numerados, o executor, o exportador do esquema, o dicionário, o diagrama e o registro textual dos resultados.

O arquivo .gitignore exclui o ambiente virtual, db.sqlite3, arquivos de mídia e temporários. Foram registrados commits por etapa, separando implementação SQL de documentação e modelagem. As alterações deste módulo foram realizadas em main. O histórico anterior contém o merge d341e4c; essa evidência é distinta de afirmar que foram utilizados branches de funcionalidade ou pull requests nesta atividade.

Quadro 3 – Registros verificáveis da evolução do projeto

Commit	Mensagem registrada	Contribuição
d112674	feat: implementacao completa do Modulo 2 - Desenvolvimento Web com Django (RF01-RF08, RNF01-RNF04)	Base funcional do MVP.
d341e4c	Merge remote-tracking branch 'origin/main'	Integração já existente no histórico.
d6a8888	feat: add public-safe demo data option	Evolução da carga de demonstração.
48431ba	feat: seed demo dogs for an existing protector	Vínculo com protetor existente.
3e91ab1	feat: implementa roteiro SQL verificavel do modulo 3	Esquema, carga, consultas, manipulação e verificações.
f3b845e	docs: documenta modelo relacional e evidencias SQL do modulo 3	Diagrama, dicionário, documentação e resultados SQL.

Fonte: histórico Git do repositório do projeto (2026).

Para revisar e registrar as alterações, foram utilizados git status, git diff, git add e git commit, com mensagens que identificam o propósito de cada etapa. O histórico pode ser consultado com git log --oneline --graph --all. A publicação no remoto utiliza git push origin main, permitindo acessar os artefatos e reproduzir a demonstração a partir do repositório.

Repositório: https://github.com/Mathuesgama/projeto_rf2

7 RESULTADOS PRELIMINARES: SOLUÇÃO INICIAL

O resultado é um modelo relacional compatível com o sistema já implementado, acompanhado de SQL executável e evidências reproduzíveis. O cenário de demonstração mantém quatro cães, com situações e responsáveis diferentes. Antes da manipulação, a vitrine retorna Paçoca e Rex; Luna fica fora por estar em tratamento, e Mel aparece na consulta de adotados. Após a adoção de Paçoca, a vitrine retorna apenas Rex, e a relação de adotados contém Paçoca e Mel.

Quadro 4 – Comparação dos resultados antes e depois do roteiro SQL

Indicador	**Antes**	**Depois**
Usuários / protetores / cães	2 / 2 / 4	2 / 2 / 4
Situação de Paçoca	disponivel	adotado
Data de adoção de Paçoca	NULL	15/10/2026
Registros de histórico	5	6
Fotografias adicionais	2	1
Cães na vitrine pública	Paçoca e Rex	Rex
Cães adotados	Mel	Paçoca e Mel
Cães não adotados por protetor	Ana: 2; Bruno: 1	Ana: 1; Bruno: 1

Fonte: execução de banco_dados/demonstrar.py com dados fictícios (2026).

As verificações rejeitaram a repetição de chave primária, um segundo perfil para o mesmo usuário, um nome nulo, a indicação de responsável inexistente e a remoção direta de um protetor com cães. O teste de rollback restaurou a situação e o histórico após uma alteração abortada. Ao final, PRAGMA foreign_key_check não apontou violações e PRAGMA integrity_check retornou ok.

Listagem 4 – Extrato da saída real da demonstração SQL

```
OK | Chaves estrangeiras ativas
OK | UPDATE: Paçoca adotado em 15/10/2026
OK | INSERT: histórico passou de 5 para 6 registros
OK | DELETE: fotos de 2 para 1; os 4 cães foram preservados
OK | ROLLBACK: situação e histórico restaurados
OK | UNIQUE: um perfil de protetor por usuário
OK | FOREIGN KEY: responsável inexistente rejeitado
OK | NOT NULL: nome nulo rejeitado
OK | PRIMARY KEY: identificador duplicado rejeitado
OK | FOREIGN KEY: protetor com cães não pode ser removido por SQL direto
OK | Nenhuma violação referencial
OK | Integridade SQLite: ok
RESULTADO: roteiro concluído com sucesso.
```

Fonte: banco_dados/evidencias_sql.txt; extrato textual da execução (2026).

Os nove testes automatizados existentes passaram com python manage.py test animais, cobrindo visibilidade pública, filtro por sexo, contato WhatsApp, exigência de login, criação de histórico, atualização de situação, obrigatoriedade da data de adoção e filtro de cães do protetor.

A conferência com makemigrations --check --dry-run não identificou divergências entre modelos e migrations.

Esses resultados demonstram a execução técnica do roteiro e a preservação dos comportamentos já testados. Não constituem avaliação de impacto social ou validação com protetores: essas atividades continuam previstas, conforme o planejamento dos módulos anteriores. Também não foram realizados testes de carga ou implantação de infraestrutura de produção nesta etapa.

8 CONCLUSÃO

O módulo consolidou a camada de persistência da aplicação Patas de Brasília, relacionando os requisitos levantados no Módulo 1 à implementação do Módulo 2. Foram documentadas as entidades, os atributos, as cardinalidades, as regras de integridade e as decisões de normalização, mantendo o escopo de apoio à adoção responsável de cães. Os scripts permitiram criar o esquema, inserir dados fictícios, consultar informações relacionadas, atualizar uma adoção com histórico e remover uma fotografia duplicada.

A execução evidenciou a importância de distinguir validações dos formulários, comportamentos do ORM e restrições efetivamente aplicadas pelo banco. O Git e o GitHub apoiaram a organização dos artefatos e a rastreabilidade por commits descritivos. Como continuidade, propõem-se a validação com protetores, o reforço de regras críticas por CHECK e transações nos fluxos da aplicação, uma política de backup e a avaliação de PostgreSQL conforme a demanda. Essas melhorias preservam a delimitação do MVP e dependem das necessidades identificadas no uso.

9 REFERÊNCIAS

CHACON, Scott; STRAUB, Ben. **Pro Git.** 2. ed. Berkeley: Apress, 2014. Disponível em: https://git-scm.com/book/en/v2. Acesso em: 15 out. 2026.

DJANGO SOFTWARE FOUNDATION. **Migrations.** Django documentation, versão 5.2. [S. l.]: Django Software Foundation, [s. d.]a. Disponível em: https://docs.djangoproject.com/en/5.2/topics/migrations/. Acesso em: 15 out. 2026.

DJANGO SOFTWARE FOUNDATION. **Model field reference.** Django documentation, versão 5.2. [S. l.]: Django Software Foundation, [s. d.]b. Disponível em: https://docs.djangoproject.com/en/5.2/ref/models/fields/. Acesso em: 15 out. 2026.

MICROSOFT. **Database normalization description.** Microsoft Learn. [S. l.]: Microsoft, [s. d.]. Disponível em: https://learn.microsoft.com/en-us/office/troubleshoot/access/database-normalization-description. Acesso em: 15 out. 2026.

SQLITE. **CREATE TABLE.** [S. l.]: SQLite, [s. d.]a. Disponível em: https://www.sqlite.org/lang_createtable.html. Acesso em: 15 out. 2026.

SQLITE. **SQLite Foreign Key Support.** [S. l.]: SQLite, [s. d.]b. Disponível em: https://www.sqlite.org/foreignkeys.html. Acesso em: 15 out. 2026.

10 LINK DO REPOSITÓRIO

Código-fonte da aplicação, migrations, documentação, modelo relacional, scripts SQL e evidências de execução:

https://github.com/Mathuesgama/projeto_rf2

Para reproduzir a atividade, clonar o repositório e executar python banco_dados/demonstrar.py. Para executar a aplicação web, seguir as instruções do README.md e instalar as dependências do projeto.